

Compilador HULK

Informe del proyecto de Compilación y Lenguajes de Programación

Arquitectura del compilador y extensiones de control de flujo

Curso de Compilación

Facultad de Matemática y Computación

Universidad de La Habana

Integrantes del equipo

Daniela De La Caridad Guerrero Álvarez

Rubén Martínez Rojas

Sammy Raul Sosa Justiz

2025 – 2026

Índice

1. Introducción	4
2. Orquestación del Pipeline y Diagnóstico de Errores	4
2.1. Función y responsabilidades	4
2.2. Modos de operación	5
2.3. El pipeline de compilación	5
2.4. El sistema de diagnósticos	5
3. El Lexer	6
3.1. Qué es un lexer y cuál es su función en el compilador	6
3.2. Por qué se eligió Flex	6
3.3. Estructura del archivo hulk_lexer.l	6
3.3.1. Opciones de configuración de Flex	6
3.3.2. Macros de patrones y el prólogo	7
3.3.3. Las reglas del lexer	7
3.4. El valor semántico y la posición	7
4. El Parser	8
4.1. Qué es un parser y cuál es su función	8
4.2. Por qué se eligió un parser LL(1)	8
4.3. Infraestructura del parser	8
4.4. La gramática LL(1) y el generador de tablas	8
4.5. El parser LL(1) predictivo y el CST	9
4.6. Nodos del AST	9
5. El Árbol de Sintaxis Abstracta y la Gestión de Identificadores	9
5.1. El módulo AST	9
5.2. El módulo SymbolTable	10
6. El Sistema de Tipos y la Representación de Valores	10
6.1. El módulo Types	10
6.2. El módulo Value	11
7. El Analizador Semántico	11
7.1. Qué es el análisis semántico y cuál es su función	11
7.2. Arquitectura general: el patrón Visitor	12
7.3. Las cuatro pasadas de analyze	12
7.4. Resolución de identificadores y comprobación de tipos	13
8. El Evaluador	13
8.1. Qué es el evaluador y cuál es su función	13
8.2. Entorno de ejecución	13
8.3. Evaluación de expresiones y estructuras de control	14
8.4. Orientación a objetos e iteración	14
9. El Generador de Código LLVM	14
9.1. Qué es la generación de código y cuál es su función	14
9.2. La cadena de herramientas y la elección de LLVM	15

9.3. Representación de valores y el protocolo de llamada uniforme	15
9.4. Layout de instancias, despacho polimórfico e identificación de tipos	15
9.5. Estructuras de control como expresiones y el runtime de C	16
10. Cumplimiento de los requisitos del proyecto	16
10.1. Requisitos técnicos	16
10.2. Extensión propia y originalidad	16
11. Extensiones de control de flujo e iteración	17
11.1. Contexto y criterios de selección	17
11.2. Definición formal frente a implementación	17
11.3. Extensión principal: <code>for</code> tipado con iteradores	17
11.3.1. Motivación y sintaxis	17
11.3.2. Cambios en gramática y AST	18
11.3.3. Semántica formal (equivalencia)	18
11.3.4. Implementación	18
11.4. Extensión secundaria: <code>repeat (n) expr</code>	19
11.4.1. Sintaxis y comportamiento	19
11.4.2. Equivalencia formal	19
11.4.3. Implementación	19
11.5. Extensión secundaria: <code>unless (cond) expr</code>	19
11.5.1. Sintaxis	19
11.5.2. Implementación	20
11.6. Extensión secundaria: <code>loop expr while (cond)</code>	20
11.6.1. Sintaxis y semántica	20
11.6.2. Implementación	20
12. Integración técnica de las extensiones	20
12.1. Archivos modificados	20
12.2. Verificaciones semánticas específicas	21
13. Comparativa con otros lenguajes	21
13.1. <code>for</code> con anotación de tipo	21
13.2. <code>repeat</code>	21
13.3. <code>unless</code>	21
13.4. <code>loop...while</code>	22
14. Crítica de HULK como diseño de lenguaje	22
14.1. Una orientación a expresiones que no está cerrada	22
14.2. Funciones globales, sin anidamiento ni cierre	23
14.3. Control de flujo: bucles sin salida y huecos de intención	24
14.4. El tipo de un <code>if</code> y el polimorfismo por herencia	25
14.5. Un solo <code>Number</code>	26
14.6. <code>=</code> y <code>:=</code>	26
15. El <code>if</code> y la cadena de operadores	27
15.1. Qué se descartó	27
15.2. Cómo se hace visible el rechazo	28
16. Decisiones de diseño y compromisos	28
16.1. Azúcar sintáctica con implementación nativa	28

16.2. Compatibilidad y uniformidad	28
16.3. Principio de menor sorpresa	28
17. Validación y pruebas	28
17.1. Estrategia general	29
17.2. Conjuntos de prueba de las extensiones	29
17.3. Objetivos del Makefile	29
18. Limitaciones y trabajo futuro	29
19. Conclusiones	30

1. Introducción

HULK (Havana University Language for Kompilers) es un lenguaje de programación diseñado con fines académicos dentro del curso de Compilación de la Facultad de Matemática y Computación de la Universidad de La Habana. El objetivo central del proyecto es construir un compilador funcional y completo para este lenguaje, recorriendo todas las fases clásicas que aparecen en la teoría de compiladores: análisis léxico, análisis sintáctico, análisis semántico y generación de código. La documentación oficial del lenguaje está disponible en <https://matcom.github.io/hulk/>.

El pipeline del compilador HULK sigue un flujo lineal y ordenado. El punto de entrada es el módulo **Compiler**, que actúa como orquestador general: recibe el código fuente, invoca las fases en el orden correcto y gestiona el diagnóstico de errores. A continuación entra en acción el **Lexer**, que transforma la secuencia de caracteres en tokens. El **Parser** toma esos tokens y construye primero un árbol de derivación concreto (CST) y luego un **AST** (Árbol de Sintaxis Abstracto), apoyándose en la **SymbolTable** para registrar identificadores. Los módulos de soporte **Types** y **Value** proveen, respectivamente, la representación estática de tipos y la representación dinámica de valores en tiempo de ejecución. Sobre el AST actúa el **Analizador Semántico**, que verifica la corrección del programa y anota cada nodo con su tipo inferido. Finalmente, el pipeline se divide en dos salidas alternativas: el **Evaluador**, que interpreta el AST directamente para producir un resultado inmediato, y el **Generador de Código LLVM**, que traduce el AST a código máquina nativo a través de LLVM IR.

El lenguaje HULK incluye expresiones aritméticas, lógicas y de comparación, estructuras de control como `if/elif/else`, `while`, `for`, `with` y `case`, declaración de funciones con y sin tipo de retorno, un sistema de tipos orientado a objetos con herencia, expresiones de ligadura con `let...in`, bloques de expresiones, operadores de verificación de tipo (`is`) y conversión (`as`), y asignación destructiva con el operador `:=`. Además, el compilador incorpora un conjunto de cuatro extensiones de control de flujo e iteración —`for` tipado, `repeat`, `unless` y `loop...while`— que constituyen el aporte propio del proyecto.

Las secciones 2 a 9 describen la arquitectura del compilador siguiendo el orden del flujo de compilación. Las secciones 10 a 13 documentan las extensiones, su integración y la comparativa con otros lenguajes. Las secciones 14 y 15 examinan HULK como diseño: qué estrecha el lenguaje, y por qué el `if` no es operando directo de una suma.

2. Orquestación del Pipeline y Diagnóstico de Errores

2.1 Función y responsabilidades

Esta capa de orquestación integra todas las fases —léxica, sintáctica, semántica y de generación de código— en una única interfaz coherente, y expone al usuario dos modos de uso distintos: un modo de producción orientado a la evaluación automática y un modo de desarrollo con flags de depuración para explorar el estado interno del compilador en cada etapa.

2.2 Modos de operación

El modo de producción (denominado internamente *matcom*) se activa cuando se pasa un archivo fuente `.hulk` sin ningún flag adicional. En este modo el compilador ejecuta el pipeline completo, reporta todos los errores encontrados en el formato `(línea,col) TIPO: mensaje` hacia la salida estándar de error, y si la compilación fue exitosa invoca la generación del ejecutable `./output` mediante LLVM.

El modo de desarrollo se activa con uno o más flags: `-tokens` imprime la lista de tokens reconocidos por el lexer, `-cst` imprime el árbol de sintaxis concreto, `-ast` imprime el árbol de sintaxis abstracto, `-symbols` imprime estadísticas de la tabla de símbolos, `-semantic` ejecuta el análisis semántico e imprime los errores encontrados, e `-interpret` ejecuta el evaluador sobre el programa. Un flag adicional, `-first-phase`, controla el comportamiento ante errores: en lugar de acumular los errores de todas las fases, se detiene en la primera fase que falla y reporta únicamente los errores de esa fase.

2.3 El pipeline de compilación

La función central del módulo es `compile_program()`, que ejecuta las fases en orden: primero tokeniza la cadena fuente y detecta errores léxicos antes de intentar el análisis sintáctico; luego construye la tabla LL(1) (con caché estático para no recalcularla en cada invocación) y ejecuta el parser, con recuperación de errores si el modo de todos los errores está activo; a continuación convierte el CST en AST; y finalmente ejecuta el análisis semántico, fusionando los errores sintácticos y semánticos si los hay antes de producir el diagnóstico final. Si no hay errores, retorna el programa compilado junto con la tabla de símbolos para que la fase de generación de código pueda consultarlos.

2.4 El sistema de diagnósticos

El sistema de diagnósticos es la infraestructura transversal que permite al módulo acumular, ordenar, deduplicar y formatear errores de cualquier fase. Cada diagnóstico almacena la posición en el fuente, el tipo de error (léxico, sintáctico o semántico) y el mensaje descriptivo. Los diagnósticos son ordenables por línea, columna y tipo, lo que garantiza que el usuario los vea en el orden natural del archivo fuente.

La función `dominant_phase()` determina la fase más grave presente en un conjunto de diagnósticos, con el orden léxico por encima del sintáctico y este por encima del semántico. El código de salida del proceso sigue directamente a esta fase: 1 para léxico, 2 para sintáctico, 3 para semántico y 0 para éxito. La función `finalize_compile_diagnostic()` ejecuta la secuencia completa de deduplicación, filtrado por modo, cálculo de fase dominante y formateo de líneas, centralizando toda la lógica en un solo lugar. Cuando el modo `-first-phase` está activo, el filtrado conserva únicamente los diagnósticos de la fase dominante; de lo contrario, todos los errores de todas las fases se presentan juntos al usuario.

En modo de producción, una compilación exitosa produce el ejecutable nativo `./output` en el directorio de trabajo, generado a partir del IR LLVM (`.hulk_out.ll`) y del runtime en C (`Codegen/runtime.c`) mediante `clang`. El binario `./hulk` es el punto de entrada del compilador; no se requiere un intérprete externo para ejecutar programas compilados.

3. El Lexer

3.1 Qué es un lexer y cuál es su función en el compilador

El lexer, también llamado analizador léxico o escáner, es la primera etapa del compilador. Su responsabilidad es leer el programa fuente como una secuencia plana de caracteres y transformarla en una secuencia de unidades con significado llamadas tokens. Un token es simplemente un par formado por un tipo y, cuando aplica, un valor semántico. Por ejemplo, el texto `42` se convierte en un token de tipo `NUMBER_LITERAL` con valor numérico 42; el texto `if` se convierte en un token de tipo `KEYWORD_IF` sin valor adicional; el texto `mensaje` se convierte en un token de tipo `IDENTIFIER` con el string `"mensaje"` como valor.

Además de producir tokens, el lexer cumple otras tareas importantes: mantiene el seguimiento de la posición actual en el archivo (número de línea y columna), descarta el ruido léxico (espacios en blanco, comentarios) que no tiene significado para el parser, y reporta errores léxicos cuando encuentra caracteres que no pertenecen al lenguaje o construcciones sin terminar como cadenas o comentarios de bloque que no se cierran.

3.2 Por qué se eligió Flex

El lexer de HULK se implementó con Flex (Fast Lexical Analyzer Generator), una herramienta estándar en el ámbito académico que genera automáticamente un analizador léxico en C++ a partir de un archivo de especificación (`hulk_lexer.l`) donde cada token se describe mediante una expresión regular y una acción. Flex fuerza al programador a pensar en los tokens según el modelo de lenguajes regulares, que es el marco teórico correcto para el análisis léxico. Además, Flex implementa la política del *longest match*: cuando varias reglas pueden aplicarse, elige la que consume más caracteres, lo que resuelve automáticamente ambigüedades como distinguir `:=` de `:` seguido de `=`. En caso de empate en longitud, Flex da prioridad a la regla que aparece primero en el archivo, lo que permite controlar la precedencia entre, por ejemplo, palabras clave e identificadores.

3.3 Estructura del archivo `hulk_lexer.l`

El archivo `hulk_lexer.l` tiene la estructura estándar de los archivos de Flex, dividida en tres secciones separadas por el marcador `%%`. La primera sección contiene las opciones de configuración de Flex y las definiciones de macros de patrones. La segunda sección, entre los dos marcadores `%%`, contiene las reglas del lexer: cada regla es un par formado por un patrón de expresión regular y el bloque de código C++ que se ejecuta cuando ese patrón hace match. La tercera sección contiene código C++ auxiliar y en este proyecto está vacía.

3.3.1 Opciones de configuración de Flex

Al comienzo del archivo se definen cuatro opciones que controlan cómo Flex genera el código del lexer. La opción `%option yyclass="HulkLexer"` le indica a Flex que genere el método `yylex()` como un método de instancia de la clase `HulkLexer` en lugar de una función global. Esto es necesario porque el proyecto está escrito en C++ orientado a objetos y se quiere que el lexer sea un objeto que se pueda instanciar con un stream de entrada específico.

La opción `%option noyywrap` evita que Flex llame a la función `yywrap()` al llegar al final del archivo. Como el lexer de HULK siempre trabaja con un solo archivo o un solo stream, esta función no es necesaria y se suprime para simplificar el código generado. La opción `%option never-interactive` le dice a Flex que trate la entrada como no interactiva, lo que permite hacer lecturas en bloque, más eficientes que leer carácter a carácter. Por último, `%option nodefault` desactiva la acción por defecto de Flex, de modo que si un carácter no hace match con ninguna regla, el programador es responsable de cubrirlo; aquí se logra con la regla de fallback `.` que reporta un error léxico explícito.

3.3.2 Macros de patrones y el prólogo

Antes de las reglas, el archivo define un conjunto de macros de patrones: alias de expresiones regulares que se pueden reutilizar dentro de las reglas reales. Las macros `DIGIT`, `ALPHA`, `ALNUM_`, `INT`, `FLOAT` e `IDENT` cubren los bloques elementales del lenguaje. Cuando una regla escribe `{FLOAT}` o `{IDENT}`, Flex sustituye automáticamente esa referencia por la expresión regular completa.

El bloque `%{ ... %}` al inicio copia código C++ al principio del archivo generado. Ahí se definen tres macros internas: `MARK_TOKEN`, que registra la posición donde comienza el token antes de que la columna avance; `SET_LEXEME`, que copia el texto reconocido a un campo del objeto; y `UPD`, que las invoca a ambas, avanza la columna y retorna el tipo del token. Esto evita repetir cuatro instrucciones en cada regla de token simple.

3.3.3 Las reglas del lexer

Los espacios, tabulaciones y retornos de carro se consumen sin generar token, actualizando solo la columna. Los saltos de línea incrementan el contador de línea y reinician la columna. Los comentarios de línea se descartan completamente. Los de bloque se manejan con un bucle manual que lee carácter a carácter y reporta error si no se encuentra el cierre.

Los literales de cadena se acumulan en un bucle que interpreta escapes y espera la comilla de cierre; un salto de línea o fin de archivo antes produce un error léxico. Los booleanos `true` y `false` se reconocen como tokens específicos antes que como identificadores, almacenando su valor en un campo booleano de la estructura semántica. Cada palabra clave tiene su propia regla ubicada antes de la regla de identificadores. Los operadores de dos caracteres se definen primero para que el *longest match* los prefiera. Al final, `«EOF»` indica el fin de la entrada y `.` cubre cualquier carácter no reconocido, reportándolo como `UNKNOWN`.

3.4 El valor semántico y la posición

Cada vez que el lexer produce un token guarda información adicional en la estructura `SemanticValue`, que tiene tres campos: `str_val` de tipo `std::string` para identificadores y cadenas, `num_val` de tipo `float` para literales numéricos, y `bool_val` de tipo `bool` para literales booleanos. La clase `HulkLexer` mantiene los campos `token_line_` y `token_col_`, que registran la posición donde comenzó el último token reconocido. Esta distinción es importante: en el momento en que `yylex()` retorna, la columna ya apunta al carácter siguiente; copiar `token_col_` en lugar de `column_` garantiza que los mensajes de error señalen el inicio del token problemático y no su carácter de cierre.

4. El Parser

4.1 Qué es un parser y cuál es su función

El parser, también llamado analizador sintáctico, es la segunda etapa del compilador. Su trabajo es tomar la secuencia de tokens producida por el lexer y verificar que esa secuencia respeta la gramática del lenguaje. Si la secuencia es válida, el parser construye una representación estructurada del programa: un árbol en el que cada nodo representa una construcción del lenguaje y los hijos de ese nodo son sus partes constituyentes.

El parser construye dos representaciones en árbol. La primera es el Árbol de Derivación Concreto (CST), que refleja exactamente la estructura de la gramática. La segunda es el Árbol de Sintaxis Abstracto (AST), que es una versión simplificada del CST, eliminando los nodos intermedios que solo existen por razones gramaticales y conservando solo la estructura semántica relevante para las fases posteriores del compilador.

4.2 Por qué se eligió un parser LL(1)

El parser implementado es de tipo LL(1): lee la entrada de izquierda a derecha, produce la derivación más a la izquierda y utiliza un solo token de lookahead para decidir qué producción aplicar. Se eligió principalmente por razones pedagógicas, ya que LL(1) conecta directamente con los conceptos teóricos del curso. Otra ventaja es que la gramática se define en un archivo separado (`grammar.ll1`) y el parser es un componente genérico que trabaja con cualquier gramática LL(1), separando claramente la descripción del lenguaje del algoritmo de análisis. La principal restricción es que la gramática no puede tener recursión izquierda ni ambigüedades, lo que obligó al equipo a diseñar una estructura gramatical más limpia y aplicar factorización.

4.3 Infraestructura del parser

Antes de poder construir el parser propiamente dicho fue necesario construir toda una infraestructura de soporte. El lexer produce tokens en su propio formato interno; para que el parser trabaje de forma limpia, se definió una estructura `Token` que agrupa el tipo del token, el texto original exacto (lexema) y la posición en el archivo. El adaptador convierte la salida del lexer a una lista de estas estructuras llamando a `yylex()` repetidamente hasta el EOF, acumulando todos los tokens en un vector. Esta separación desacopla el parser de la API interna de Flex.

La clase `TokenStream` mantiene el vector de tokens y un índice que apunta al token actual. El método más importante es `consume(type, msg)`: verifica que el token actual sea del tipo esperado y, si no lo es, lanza un `ParseError` con el mensaje descriptivo y la posición del token problemático. La clase `ParseError` extiende `std::exception` y almacena el token problemático con toda la información necesaria para reportar el error al usuario.

4.4 La gramática LL(1) y el generador de tablas

La gramática formal del lenguaje HULK vive en el archivo `grammar.ll1`. Este archivo es la fuente de verdad sobre la sintaxis del lenguaje y el parser no tiene conocimiento codificado de ella; en cambio, la lee y genera la tabla LL(1) a partir de ella en tiempo de ejecución.

Para que la gramática sea apta para LL(1) fue necesario eliminar toda recursión izquierda. El ejemplo más representativo es la precedencia de operadores: la producción natural $\text{Expr} \rightarrow \text{Expr} + \text{Expr}$ se transforma en el patrón estándar que introduce un no terminal “cola”, convirtiendo la recursión izquierda en recursión derecha más un caso base épsilon. Para la potencia, que necesita asociatividad a la derecha, el no terminal cola llama de vuelta a la producción completa, de modo que 2^{3^2} se interpreta correctamente como $2^{(3^2)}$.

El generador de tablas calcula los conjuntos FIRST y FOLLOW de forma iterativa hasta alcanzar un punto fijo, y construye la tabla de análisis predictivo $M[A, a]$. Si en algún momento se intenta agregar una segunda producción a una celda que ya tiene una, hay un conflicto LL(1) y la gramática no es apta; en el estado actual del proyecto, la tabla no tiene ningún conflicto.

4.5 El parser LL(1) predictivo y el CST

El parser LL(1) implementado es un analizador predictivo dirigido por la tabla. Su algoritmo mantiene una pila que inicialmente contiene el símbolo inicial **Program**. En cada paso mira el tope de la pila y el token actual: si el tope es un terminal y coincide, ambos se emparejan; si el tope es un no terminal, consulta la tabla y apila los símbolos del lado derecho en orden inverso. Si la celda está vacía, reporta un error sintáctico. Durante este proceso se construye simultáneamente el CST.

El CST no es adecuado para el análisis semántico directamente porque contiene mucho ruido: nodos que representan producciones épsilon, nodos de no terminales auxiliares como **AddExprTail** que no tienen significado semántico propio, y múltiples niveles de anidamiento consecuencia de la factorización LL(1). Por eso se construye el AST mediante un recorrido recursivo del CST que elimina ese ruido y preserva solo la estructura semántica.

4.6 Nodos del AST

El AST se organiza bajo un nodo raíz **Program** con dos jerarquías. Las sentencias representan componentes de nivel superior: declaraciones de funciones, de tipos o expresiones con punto y coma. Las expresiones representan cualquier construcción que produce o evalúa un valor, cubriendo literales, operaciones binarias y unarias, estructuras de control, ligaduras **let**, bloques, llamadas a función, accesos a atributos, instanciación de tipos, **with**, **case** y las extensiones **unless**, **repeat** y **loop...while**, además de **ForExpr** con anotación de tipo opcional (**declared_type**).

Al convertir el CST a AST, la función de conversión normaliza estructuras complejas. Las cadenas de nodos de operaciones aritméticas se aplanan en árboles de nodos binarios con asociatividad izquierda. Las operaciones postfix (accesos y llamadas encadenadas como **obj.m(a).b**) se procesan de izquierda a derecha para construir estructuras jerárquicas correctas. Los **elif** se normalizan recursivamente en nodos **if** anidados.

5. El Árbol de Sintaxis Abstracta y la Gestión de Identificadores

5.1 El módulo AST

El módulo AST actúa como capa de adaptación entre el namespace interno del parser y el resto del compilador. Su propósito es exponer en el namespace global todos los nodos del ár-

bol de sintaxis abstracta y las interfaces del patrón Visitor, permitiendo que los módulos de análisis semántico, verificación de tipos y evaluación los utilicen sin necesidad de calificar explícitamente el namespace `parser::`. El módulo no define nuevas estructuras de datos propias: delega completamente en los nodos declarados en el parser, manteniendo una única fuente de verdad para la representación del programa. Tres nodos reciben alias con nombres más descriptivos: `BooleanExpr` para `parser::BoolExpr`, `VariableExpr` para `parser::IdentifierExpr` y `ExprBlock` para `parser::BlockExpr`. El resto se reexportan sin cambio de nombre mediante declaraciones `using`.

5.2 El módulo `SymbolTable`

El módulo `SymbolTable` implementa la tabla de símbolos del compilador: la estructura de datos que registra y resuelve todos los identificadores que aparecen en el programa fuente. Opera durante el análisis semántico y es consultada por el verificador de tipos, el analizador semántico y el módulo `Types` para resolver referencias y verificar conformidad de herencia.

La tabla mantiene tres tipos de símbolos. Los símbolos de variable almacenan el nombre del identificador, su tipo estático, si es mutable y la línea de declaración. Los símbolos de función almacenan la firma completa y soportan sobrecarga por aridad, organizando múltiples definiciones del mismo nombre en un vector. Los símbolos de tipo representan una clase con su jerarquía, atributos y métodos.

La gestión de ámbitos se implementa mediante una pila donde cada entrada es un mapa de variables del ámbito activo. `pushScope()` abre un nuevo ámbito anidado y `popScope()` lo cierra, descartando todas las variables declaradas en él. El shadowing está explícitamente permitido: un ámbito hijo puede declarar la misma variable que un ámbito padre, y la búsqueda resolverá al símbolo más interno. Las funciones se almacenan fuera de esta pila porque en el lenguaje tienen alcance global. Los tipos también se manejan en un mapa separado, con búsqueda recursiva en la cadena de herencia para resolver atributos y métodos heredados.

El constructor inicializa el entorno con todo lo predefinido: las constantes `PI` y `E` como variables inmutables, la jerarquía de tipos base (`Object`, `Number`, `String`, `Boolean`, `Null`, `Void`, `Unknown`, `Iterable`), y las funciones builtin (`sin`, `cos`, `sqrt`, `exp`, `log`, `rand`, `print` y `range`). Los builtins se protegen de redeclaración por el usuario.

El recolector de declaraciones `DeclCollector` implementa una primera pasada superficial sobre el AST que registra en la tabla todas las funciones y tipos de primer nivel antes de que comience el análisis semántico completo, habilitando referencias hacia adelante. Los parámetros se registran con tipo desconocido intencionalmente: solo se necesita saber que la función existe y con qué aridad.

6. El Sistema de Tipos y la Representación de Valores

6.1 El módulo `Types`

El módulo `Types` provee la representación estática de tipos usada durante el análisis semántico. Su clase central, `TypeInfo`, encapsula tanto los tipos primitivos del lenguaje (`Number`, `String`, `Boolean`, `Function`, `Null`, `Void`, `Unknown`) como los tipos nominales definidos por el usuario,

identificados por el nombre de la clase. El valor **Unknown** es central en el diseño: representa un tipo que todavía no se ha podido determinar y se usa como comodín durante la inferencia.

El sistema de conformidad de tipos se implementa en el método `conformsTo(other)`, que aplica reglas en orden de prioridad: si cualquiera de los dos tipos es **Unknown**, la conformidad es verdadera para soportar la inferencia gradual; si ambos tienen el mismo `kind` y nombre, son idénticos; **Null** conforma a cualquier tipo objeto; **Object** sin nombre es supertipo universal de todo; y para pares de tipos nominales se sigue la cadena de herencia mediante `isSubtypeOf`, que recorre la jerarquía usando la tabla de símbolos para resolver cada `ClassDecl`.

El cálculo del mínimo ancestro común (LCA) se usa en el verificador de tipos para unificar las ramas de un `if/else` o un `case`: el método estático `getLowestCommonAncestor` combina una lista de tipos encontrando el supertipo más específico que es común a todos. Para pares de tipos nominales, el algoritmo construye el conjunto de todos los ancestros del primer tipo y luego recorre la cadena del segundo buscando el primer nombre en ese conjunto.

La inferencia estática de tipos para operadores se implementa en `inferBinaryOp` e `inferUnaryOp`, que determinan el tipo resultante de una operación a partir de los tipos de los operandos sin necesidad de conocer sus valores concretos.

6.2 El módulo Value

El módulo `Value` define la representación en tiempo de ejecución de todos los valores que puede producir o manipular el intérprete. Su clase central, `Value`, utiliza `std::variant` como almacenamiento interno, lo que garantiza que solo uno de los tipos alternativos esté activo en cada momento y que la verificación sea exhaustiva en tiempo de compilación. Las alternativas son: `monostate` para **Null**, `double` para **Number**, `string` para **String**, `bool` para **Boolean**, y punteros compartidos para rangos, iteradores de rango e instancias de clase de usuario.

La clase expone dos grupos de métodos simétricos: predicados que verifican qué alternativa está activa y accesores que extraen el valor lanzando excepción si el tipo no coincide. Esta simetría garantiza que el evaluador siempre verifique el tipo antes de acceder al valor.

Las instancias de clases de usuario se representan mediante la estructura `Instance`, que combina un marco de entorno para los atributos (cuyo padre es el entorno global en el momento de creación), un puntero al nodo `ClassDecl` del AST para búsqueda de métodos y verificación de herencia, y un puntero circular a la propia instancia para pasar `self` a los métodos sin crear copias.

Para la iteración, `RangeValue` modela el rango semiabierto $[start, end)$ producido por `range(start, end)`, y `RangeIterator` implementa el protocolo de iteración nativa. El método `next()` inicializa el iterador en la primera llamada y avanza en 1.0 en las sucesivas, retornando falso cuando se alcanza el extremo del rango.

7. El Analizador Semántico

7.1 Qué es el análisis semántico y cuál es su función

El análisis semántico es la tercera etapa del compilador y recibe como entrada el AST construido por el parser. Mientras que el parser solo verifica que el programa esté bien formado según la

gramática, el análisis semántico verifica que el programa tenga sentido: que toda variable usada haya sido declarada antes, que toda función o método se invoque con la cantidad correcta de argumentos, que los tipos de las expresiones sean compatibles entre sí, que las clases formen una jerarquía de herencia válida y sin ciclos, y que las redefiniciones de nombres no entren en conflicto con declaraciones anteriores.

El componente que implementa esta fase se llama **Phase2Analyzer**. El nombre refleja una decisión de diseño: el análisis semántico de HULK no se hace en una sola pasada lineal sobre el AST, sino en varias fases sucesivas, necesarias porque el lenguaje permite referencias hacia adelante e inferencia de tipos mutuamente dependiente.

7.2 Arquitectura general: el patrón Visitor

Phase2Analyzer hereda simultáneamente de `parser::ExprVisitor` y `parser::StmtVisitor`, de modo que para cada tipo de nodo del AST implementa un método `visit` que recibe un puntero al nodo concreto. El estado que se propaga entre llamadas no se pasa como parámetro de retorno explícito, sino a través de dos miembros privados: `current_type_`, que guarda el tipo inferido de la última expresión visitada, y `current_class_`, que guarda el nombre de la clase dentro de cuyo método se está analizando en este momento. El método auxiliar `visitExpr` no solo invoca el patrón visitor sino que además registra en un mapa el tipo resultante; esto garantiza que cada nodo del AST quede anotado con su tipo inferido, lo cual es un prerequisite para la generación de código LLVM.

7.3 Las cuatro pasadas de analyze

El método público `analyze(Program*)` organiza el trabajo en pasadas sucesivas. En la primera pasada, `collectClassDeclarations` y `collectFunctionDeclarations` recorren el programa registrando en la tabla de símbolos el nombre de cada clase y cada función antes de analizar el cuerpo de ninguna de ellas, lo que habilita las referencias hacia adelante. En la segunda pasada se visita explícitamente cada declaración de clase para validar la jerarquía de herencia, detectar ciclos, registrar atributos y métodos, y aplicar la regla de herencia implícita de parámetros cuando una subclase no declara los propios.

La tercera pasada, la más particular, implementa la inferencia de tipos hasta el punto fijo. Cuando un parámetro de función no tiene tipo declarado, su tipo inicial es **Unknown**. Al analizar el cuerpo, los operadores que detectan un operando desconocido y el otro concreto propagan el tipo hacia atrás a la variable involucrada. Después de visitar el cuerpo, se actualiza la firma registrada de la función con los tipos que terminaron teniendo sus parámetros. Como esta inferencia puede depender de otra función todavía no analizada, el analizador repite el recorrido completo hasta diez veces, deteniéndose tan pronto como una iteración completa no produce ningún cambio. Este es el algoritmo de punto fijo: el proceso converge porque el conjunto de tipos posibles es finito y la inferencia solo se mueve de **Unknown** hacia un tipo concreto, nunca al revés. La cuarta y última pasada valida que cuando un método sobrescribe a uno heredado, su tipo de retorno conforme al del método base, comprobación que se posterga porque los tipos de retorno pueden seguir siendo provisionales mientras dura la fase de punto fijo.

7.4 Resolución de identificadores y comprobación de tipos

El analizador valida cada uso de variable consultando la tabla de símbolos y reportando error si no existe en ningún ámbito visible. Para las funciones distingue dos motivos de error: si el nombre existe pero con distinta aridad, reporta un error de cantidad de argumentos; si no existe con ninguna aridad, reporta función no definida. Esta distinción también es la base de la sobrecarga por aridad, que el lenguaje permite siempre que dos funciones del mismo nombre difieran en la cantidad de parámetros.

La comprobación de tipos en operadores binarios es el método más largo del analizador porque la validez depende del operador. Los operadores aritméticos exigen `Number` en ambos lados; los lógicos exigen `Boolean`; las comparaciones de orden exigen `Number`; la igualdad exige que ambos lados tengan el mismo `kind`; la concatenación exige `String`. Antes de cada comprobación, el analizador intenta propagar el tipo del operando concreto hacia el operando desconocido, actualizándolo en la tabla de símbolos.

Las estructuras de control que pueden aparecer como expresiones producen su tipo mediante la unificación con el mínimo ancestro común: si el `then` devuelve una instancia de `Perro` y el `else` devuelve una instancia de `Gato`, ambas subclases de `Animal`, el tipo del `if` completo es `Animal`. El analizador implementa además comprobaciones específicas para la herencia de clases, las llamadas al constructor de la clase base, la inicialización de atributos y la sobreescritura compatible de métodos.

8. El Evaluador

8.1 Qué es el evaluador y cuál es su función

El módulo Evaluador constituye la Fase 3 del compilador: un intérprete de árbol de sintaxis abstracta que recorre el programa ya parseado y produce directamente su resultado en tiempo de ejecución. Opera sobre el AST generado por las fases anteriores y materializa la semántica del lenguaje sin pasar por generación de código intermedio. Al igual que el analizador semántico, implementa el patrón Visitor para expresiones y sentencias.

8.2 Entorno de ejecución

El entorno de ejecución se modela con la estructura `EnvFrame`, que implementa una cadena de marcos léxicos. Cada frame contiene un mapa de nombre a valor y un puntero opcional al frame padre, lo que permite recorrer la cadena léxica en la búsqueda de variables y soporta correctamente el alcance léxico, los bloques anidados y los cierres capturados por las funciones de usuario.

La evaluación de un programa completo se realiza en dos pasadas: en la primera se registran todas las funciones y clases (permitiendo referencias hacia adelante), y en la segunda se ejecutan las sentencias que no son declaraciones. Antes y después de cada bloque de ejecución que requiere su propio alcance, el evaluador salva y restaura el frame activo. Esta garantía de limpieza de entorno también se aplica en presencia de errores, ya que la propagación de errores corta la evaluación antes de continuar.

8.3 Evaluación de expresiones y estructuras de control

Los literales se convierten directamente al tipo **Value** correspondiente. Los operadores lógicos implementan cortocircuito: si el operando izquierdo ya determina el resultado, el derecho no se evalúa. Los bloques evalúan sus subexpresiones en secuencia y el valor resultante es el de la última. La ligadura **let** crea un nuevo frame hijo, define la variable con el valor del inicializador, evalúa el cuerpo en ese frame y restaura el frame anterior.

Las estructuras de control cubren las variantes **if/unless**, **while**, **repeat**, **loop...while** y **for**. El **for** soporta tanto el rango nativo producido por **range(start, end)** como el protocolo de iteración definido por el usuario, que exige que el objeto implemente los métodos **next()** y **current()**. La expresión **with** maneja valores potencialmente nulos de forma segura: si el valor es **Null**, ejecuta la rama **else**; de lo contrario, enlaza el alias al valor y evalúa el cuerpo. El **case** implementa despacho por tipo buscando la rama cuyo tipo más concreto en la jerarquía de herencia sea compatible con el valor del escrutinio.

8.4 Orientación a objetos e iteración

La construcción **new Tipo(args)** localiza la **ClassDecl** en el registro de tipos, verifica la aridad, crea una instancia con su propio frame de atributos, e inicializa en orden los parámetros del constructor, los atributos heredados del padre (de forma recursiva) y los atributos propios de la clase. La búsqueda de métodos también es recursiva por la cadena de herencia. La invocación de método guarda y restaura el estado relevante, crea un frame hijo con los parámetros y evalúa el cuerpo con **self** declarado. La llamada **base(args)** permite invocar el método del mismo nombre en la clase padre sin cambiar la instancia activa.

El evaluador adopta una estrategia de error sin excepciones: el método **setError()** activa una bandera y almacena el mensaje, y todos los métodos comprueban esta bandera antes de proceder, cortocircuitando la evaluación al primer error. Esto produce un comportamiento predecible donde siempre se reporta el error más temprano en el flujo de ejecución.

9. El Generador de Código LLVM

9.1 Qué es la generación de código y cuál es su función

La generación de código es la cuarta y última fase del compilador. Recibe el AST ya validado por el analizador semántico y produce un artefacto ejecutable. A diferencia del intérprete, que recorre el AST y evalúa directamente cada nodo en tiempo de ejecución, esta fase traduce el AST a LLVM IR (Intermediate Representation), una representación de bajo nivel, tipada y en forma casi-ensamblador que la infraestructura de LLVM sabe optimizar y compilar a código máquina nativo. El resultado final es un binario ELF de Linux x86_64 capaz de ejecutarse de forma independiente.

El componente vive en el directorio **Codegen/** y su clase principal es **LLVMCodeGenerator**, que hereda de los mismos visitantes del AST que el analizador semántico, reutilizando exactamente el mismo patrón de recorrido. Lo único que cambia es qué se construye en cada **visit**: en lugar de un **TypeInfo**, cada visita a una expresión deja un **llvm::Value***, el handle que la API de LLVM usa para referirse a un valor o instrucción dentro del IR.

9.2 La cadena de herramientas y la elección de LLVM

El generador produce texto LLVM IR, que luego se escribe a un archivo temporal `.hulk_out.ll` y se pasa a `clang` junto con la biblioteca de soporte en C `runtime.c` para producir el ejecutable final. Esta elección está motivada por razones prácticas: LLVM es la infraestructura de compilación estándar de la industria, provee una API de C++ madura para construir IR de manera programática, verificación automática del IR generado y un ecosistema completo de optimización, ninguno de los cuales el equipo necesitó reimplementar.

9.3 Representación de valores y el protocolo de llamada uniforme

La decisión de diseño más importante es cómo representar los distintos tipos de valor del lenguaje a nivel de IR, dado que LLVM no tiene noción de valor dinámicamente tipado. El generador usa una estrategia mixta: los valores `Number` se representan como `double` nativo, los `Boolean` como `i1` (un solo bit), y los `String` y cualquier valor cuyo tipo no se conoce en tiempo de generación se representan mediante un puntero a una estructura `BoxedValue` reservada en el heap. Esta estructura “etiquetada” de tamaño fijo tiene un campo entero de etiqueta y un área de 8 bytes donde se almacena el dato real, y debe coincidir exactamente entre la definición de LLVM y la del runtime de C.

Para que funciones y métodos no necesiten conocer el tipo dinámico exacto de cada valor, se adopta una convención de llamada uniforme: toda función de usuario y todo método siempre recibe y devuelve un puntero opaco desde el punto de vista de la firma LLVM. Antes de cada llamada, los valores nativos se empaquetan en `BoxedValue` si es necesario; al volver, el resultado se desempaqueta al tipo nativo si el tipo de retorno esperado lo requiere. Esta uniformidad evita una combinatoria de variantes de función por cada combinación de tipos.

9.4 Layout de instancias, despacho polimórfico e identificación de tipos

Para cada clase se construye un `llvm::StructType` siguiendo el principio de prefijo común: si una clase tiene una clase base, la función primero garantiza que el struct de la base esté construido y luego copia su lista de campos como prefijo antes de añadir los campos propios. El primer campo de toda clase raíz es un campo especial que almacena el tipo en tiempo de ejecución de la instancia como un puntero al nombre de la clase registrado una sola vez como constante global. Como las subclasses heredan este campo, toda instancia tiene su tipo dinámico siempre en el mismo desplazamiento, lo que permite implementar `is`, `as` y el despacho polimórfico sin una tabla de métodos virtuales.

El despacho polimórfico se implementa como una cadena de comparaciones de tipo en tiempo de ejecución construida en el propio punto de llamada: para cada clase del programa que implementa el método solicitado, se emite una comparación del tipo dinámico de la instancia contra el nombre de esa clase, y si coincide, se invoca la versión específica de esa clase. Todos los caminos exitosos confluyen mediante una instrucción `phi`. Esta estrategia logra el mismo efecto observable que una `vtable` pero es considerablemente más simple de implementar y de depurar, pagando el costo con comparaciones lineales en la cantidad de clases candidatas en lugar de una sola indirección de puntero.

9.5 Estructuras de control como expresiones y el runtime de C

Al igual que en el analizador semántico, las construcciones `if`, `unless`, `case` y `with` deben producir un valor LLVM combinado de todas sus ramas. La herramienta estándar de LLVM para esto es la instrucción `phi`: se crean los bloques básicos necesarios, se emite la condición y un salto condicional, cada rama termina con un salto al bloque de confluencia, y en ese bloque una instrucción `phi` selecciona el resultado correcto según desde qué bloque se llegó. Los bucles `while`, `repeat` y `loop...while` no pueden usar `phi` directamente porque la cantidad de iteraciones no se conoce en tiempo de compilación; en su lugar, reservan celdas de pila que acumulan el último valor producido y se actualizan en cada iteración.

Los operadores lógicos con cortocircuito tampoco se traducen como instrucciones binarias simples: el cortocircuito debe quedar codificado explícitamente como control de flujo dentro del IR, con un bloque que solo se genera si el operando izquierdo no determina el resultado por sí solo.

Una parte deliberada del comportamiento del programa compilado se delega a una pequeña biblioteca de soporte en C, `runtime.c`, que se compila y enlaza junto con el IR generado. Esta biblioteca cubre el empaquetado y desempaquetado de valores, las operaciones sobre cadenas (concatenación, comparación de igualdad), las funciones de impresión y el reporte de errores en tiempo de ejecución. La separación mantiene el generador de código enfocado en la traducción estructural del AST mientras que la lógica de manejo de memoria y formato de cadenas se escribe una sola vez en un lenguaje más simple de mantener.

10. Cumplimiento de los requisitos del proyecto

10.1 Requisitos técnicos

El compilador cumple las restricciones del enunciado en los siguientes términos:

- Lenguaje de implementación: C++17. La lógica del evaluador y del flujo principal evita excepciones para reportar errores; se usan banderas y códigos de salida.
- Fase frontal: analizador léxico generado con Flex (`Lexer/hulk_lexer.1`) y analizador sintáctico LL(1) con gramática en `Parser/grammar/grammar.ll1`.
- Generación de código principal: salida nativa mediante representación intermedia de LLVM y `clang`.
- Motor de ejecución adicional: intérprete que recorre el árbol sintáctico abstracto (`Evaluator/evaluator.cpp`) activable con `-interpret` y usado como referencia en las pruebas.
- Entrega: repositorio con `make build` → `./hulk`, diagnósticos (`línea,col`) TIPO: mensaje y generación de `./output`.

10.2 Extensión propia y originalidad

El requisito de al menos una característica adicional con cambios de sintaxis y semántica se cumple con un conjunto coherente de cuatro extensiones de control de flujo e iteración. La extensión principal —y la que introduce verificación estática nueva— es `for (x : T in expr)`, que añade anotación de tipo opcional en la cabecera del bucle. Las tres restantes (`repeat`, `unless`, `loop...while`) amplían la expresividad sin modificar el modelo computacional.

Para diferenciar el trabajo respecto a otros equipos que implementen construcciones similares, el diseño destaca: (i) anotación de tipo opcional en el `for`, coherente con el tipado gradual; (ii) sintaxis `loop expr while (cond)` en lugar de `do...while`; (iii) integración directa en gramática LL(1), árbol sintáctico abstracto y tres motores de ejecución (verificación semántica, intérprete y LLVM) sin colisión con identificadores del lenguaje base.

11. Extensiones de control de flujo e iteración

11.1 Contexto y criterios de selección

Antes de las extensiones, el compilador ya soportaba `if-elif-else`, `while`, `for` sobre `range` y sobre objetos con métodos `next()` y `current()`, además de `let-in`, bloques y el sistema de tipos con herencia. La gramática LL(1) en `grammar.ll1` impone que toda nueva producción sea desambiguada con un solo símbolo de anticipación.

Las extensiones se eligieron por reutilización de infraestructura, coherencia temática (control de flujo e iteración), gradación de complejidad (una extensión principal y tres secundarias) y comparabilidad con lenguajes conocidos (Ruby, Kotlin, C#, C).

11.2 Definición formal frente a implementación

Es importante distinguir dos niveles:

1. Semántica formal: cada extensión se define por equivalencia con construcciones base (`if`, `while`, `let`). Estas equivalencias son la referencia para razonar sobre el comportamiento.
2. Implementación en el compilador: el flujo de compilación no incluye una fase de desazucarado del árbol sintáctico abstracto. Las extensiones producen nodos dedicados (`UnlessExpr`, `RepeatExpr`, `LoopWhileExpr`, `ForExpr` con `declared_type` opcional) que se procesan mediante visitas en el analizador semántico, el evaluador y el generador LLVM.

Esta decisión mantiene el árbol legible en diagnósticos (`-ast`), permite optimizaciones específicas (por ejemplo, `for` sobre `range`) y conserva paridad entre intérprete y código nativo.

11.3 Extensión principal: `for` tipado con iteradores

11.3.1 Motivación y sintaxis

El lenguaje ya disponía de una forma básica de `for` que permitía iterar sobre cualquier expresión que implementase el protocolo `Iterable`, pero en esa forma original la variable de iteración no podía recibir una anotación de tipo explícita: el programador dependía completamente de la inferencia o recurría a conversiones manuales con `as`. La motivación de la extensión tiene además una dimensión semántica: al admitir la anotación, el verificador de tipos puede comprobar estáticamente que el tipo devuelto por `current()` del iterador conforma al tipo declarado, en lugar de diferir esa verificación.

La forma base `for (x in expr) body` ya existía. La extensión añade la variante opcional:

```
for (x : T in expr) body
```

La anotación permite legibilidad explícita y verificación estática de que el elemento iterable conforma al tipo declarado, sin obligar al programador a abandonar la inferencia cuando no la necesita.

11.3.2 Cambios en gramática y AST

En `grammar.ll1`:

```
ForExpr    -> FOR LPAREN IDENTIFIER ForTypeOpt IN Expr RPAREN ForBody
ForTypeOpt -> COLON IDENTIFIER | epsilon
```

El nodo `ForExpr` almacena `variable`, `declared_type` (opcional), `iterable` y `body`. La conversión del árbol concreto al abstracto está en `Parser/ast/cst_to_ast.cpp`.

11.3.3 Semántica formal (equivalencia)

La referencia del lenguaje transpila el `for` a un `while` con iterador auxiliar:

```
let _iter = expr in while (_iter.next()) let x [: T] = _iter.current() in body
```

11.3.4 Implementación

`Phase2Analyzer::visit(ForExpr)` (`SemanticCheck/phase2_checker.cpp`):

- verifica que el iterable sea `range` o un objeto con `next()` que devuelve `Boolean` y `current()`;
- infiere el tipo del elemento; si hay `declared_type`, exige que el tipo del elemento conforme al declarado;
- declara la variable de iteración en un ámbito hijo y analiza el cuerpo.

El evaluador distingue `RangeValue/RangeIterator` (camino optimizado) de iteradores definidos por el usuario (llamadas a `next()/current()`). LLVM emite un bucle numérico directo para `range` y un bucle con llamadas a métodos para iteradores personalizados (`Codegen/llvm_codegen.cpp`).

Ejemplo de uso:

```
for (x: Number in range(0, 3)) print(x);
for (w: String in new WordIter()) print(w);
```

Justificación del diseño. Mantener la anotación como optativa, en lugar de obligatoria, responde a la filosofía de tipado gradual de HULK: un programador que confía en la inferencia no necesita cambiar nada, mientras que quien trabaja con jerarquías de tipos complejas puede ser explícito sin coste adicional. Representar el tipo anotado como campo opcional en `ForExpr` evita añadir complejidad estructural al verificador: cuando el campo es nulo, el tipo del elemento se infiere normalmente; cuando está presente, se añade únicamente una comprobación de conformidad que reutiliza la infraestructura ya existente (`conformsTo`).

11.4 Extensión secundaria: `repeat (n) expr`

11.4.1 Sintaxis y comportamiento

Uno de los patrones más frecuentes en programación es repetir una acción un número fijo y conocido de veces. En HULK base esto se expresaba mediante un `while` con contador explícito o un `for` sobre un rango; ambas formas son correctas pero ninguna expresa directamente la intención de “*que algo suceda n veces*”. `repeat` cubre ese caso de uso con una sintaxis directa e inmediatamente legible.

```
repeat (n) expr
```

Donde `n` debe ser de tipo `Number`. El cuerpo se evalúa un número de veces igual a la parte entera de `n` (truncación hacia cero). Si $n \leq 0$, el resultado es `Null` sin ejecutar el cuerpo; en caso contrario, el valor de la expresión es el de la última ejecución del cuerpo.

11.4.2 Equivalencia formal

```
let _total = n in while (_total > 0) { _total := _total - 1; expr }
```

11.4.3 Implementación

Gramática: `RepeatExpr -> REPEAT LPAREN Expr RPAREN Expr`. Nodo `RepeatExpr{count, body}`. Palabra clave `repeat` en `hulk_lexer.l`. Visitas dedicadas en `phase2_checker.cpp`, `evaluator.cpp` (bucle con contador entero) y `llvm_codegen.cpp` (contador `i32` con rama para valores no positivos).

Justificación del diseño. Modelar `repeat` con referencia semántica a `while` en lugar de a `for` sobre un rango obedece a que la intención es distinta: `repeat` no itera sobre elementos, sino que cuenta ejecuciones. Usar `for` introduciría una variable de iteración que el programador no necesita y que podría inducir confusión; el contador descendente en `while` es más fiel a la intención de la construcción.

11.5 Extensión secundaria: `unless (cond) expr`

11.5.1 Sintaxis

La construcción condicional negada es un patrón frecuente en código imperativo: ejecutar algo si y solo si una condición *no* se cumple. La forma canónica `if (!cond) expr` reduce la legibilidad cuando la condición es larga o compleja, obligando al lector a invertir mentalmente la negación; `unless` elimina ese salto y expresa directamente la intención.

```
unless (cond) expr      unless (cond) expr else alt
```

Semánticamente equivalente a `if (!cond) expr [else alt]`. No admite `elif` por diseño: casos complejos deben usar `if`.

11.5.2 Implementación

Terminal **UNLESS** en el analizador léxico. Nodo `UnlessExpr{condition, then_branch, else_branch}`. El verificador semántico exige condición `Boolean` y unifica tipos de ramas mediante el mínimo ancestro común. En LLVM se genera un grafo de flujo de control propio con bloques `unless.then/unless.else` y fusión de valores. Caso límite: sin `else` y condición verdadera, el evaluador produce 0 (comportamiento verificado en `unless_no_else_true.hulk`).

Justificación del diseño. La decisión más relevante en el diseño de `unless` fue no admitir `elif`. Esta restricción evita que `unless` se convierta en un duplicado complejo de `if` con lógica invertida; la construcción se mantiene simple y de propósito único, mientras que cualquier lógica condicional más elaborada sigue siendo territorio de `if`.

11.6 Extensión secundaria: `loop expr while (cond)`

11.6.1 Sintaxis y semántica

Los bucles `while` tradicionales evalúan la condición antes de ejecutar el cuerpo: si es falsa desde el inicio, el cuerpo nunca se ejecuta. Hay, sin embargo, situaciones —lectura de entradas, procesamiento de flujos de datos, algoritmos que siempre realizan al menos una iteración— donde se desea ejecutar el cuerpo al menos una vez antes de comprobar la condición. `loop expr while (cond)` cubre ese patrón postcondicional de forma clara y declarativa.

`loop expr while (cond)`

Bucle postcondicional (equivalente al `do...while` de C): el cuerpo se ejecuta al menos una vez; mientras `cond` sea verdadera, se repite. El valor resultante es el de la última ejecución del cuerpo. La condición debe ser `Boolean`.

11.6.2 Implementación

Gramática: `LoopWhileExpr -> LOOP Expr WHILE LPAREN Expr RPAREN`. Nodo `LoopWhileExpr{body, condition}`. El evaluador ejecuta el cuerpo y luego evalúa la condición en un bucle; LLVM usa bloques `loopwhile.body` y `loopwhile.cond`.

Justificación del diseño. La elección de la palabra clave `loop` en lugar de la forma `do...while` de C y Java fue deliberada: `do...while` introduce dos palabras clave y rompe la convención visual de HULK, donde los bloques se identifican por llaves o por la posición del cuerpo respecto a la condición. `loop...while` es más uniforme con la gramática existente y sitúa la condición al final, de modo que el lector sabe desde el inicio que está ante un bucle cuyo cuerpo precede a su condición de corte.

12. Integración técnica de las extensiones

12.1 Archivos modificados

- Análisis léxico: `Lexer/hulk_lexer.1` (palabras clave `repeat`, `unless`, `loop`).

- Gramática: `Parser/grammar/grammar.ll1`.
- Árbol sintáctico abstracto y conversión: `Parser/ast/expr.hpp`, `cst_to_ast.cpp`.
- Verificación semántica: `SemanticCheck/phase2_checker.cpp`.
- Intérprete: `Evaluator/evaluator.cpp`.
- Generación de código: `Codegen/llvm_codegen.cpp`.
- Pruebas: `tests/extensions/`, `tests/eval/`, `tests/llvm/`, `tests/semantic/`.

12.2 Verificaciones semánticas específicas

- **unless**: condición booleana; tipo del resultado = mínimo ancestro común de las ramas.
- **repeat**: contador de tipo `Number`; tipo del resultado = tipo del cuerpo.
- **loop...while**: condición booleana; tipo del resultado = tipo del cuerpo.
- **for** tipado: iterable válido; `current()` compatible con `T` si se anota.

La gramática LL(1) resultante no presenta conflictos predictivos (verificado por pruebas del generador de tablas).

13. Comparativa con otros lenguajes

13.1 for con anotación de tipo

Kotlin (`for (x: T in xs)`) y C# (`foreach (T x in collection)`) son los precedentes más cercanos. Python y Rust infieren el tipo en la cabecera; en Rust el compilador lo hace con precisión gracias a su sistema de rasgos, conceptualmente similar al sistema de protocolos de HULK. La diferencia principal frente a C# es la optatividad: en C# la anotación es obligatoria (o reemplazable por `var` para inferencia), mientras que en HULK la anotación es siempre opcional, preservando la flexibilidad del tipado gradual.

13.2 repeat

Ruby usa `n.times { ... }` como método del número; HULK adopta **repeat** como palabra clave de primer orden del lenguaje en lugar de un método sobre un objeto numérico, lo cual evita comprometerse con la idea de que los enteros sean objetos con métodos. Pascal y bucles **for** numéricos cubren el mismo caso con más verbosidad. Lua tiene un `repeat...until` con semántica distinta (postcondicional, opuesta al **while**), que no debe confundirse con la construcción de HULK. En lenguajes educativos como Logo o Scratch, la repetición acotada directa es casi ubicua y suele ser la primera estructura de control que aprenden los estudiantes, lo que refuerza su valor pedagógico en un lenguaje didáctico como HULK.

13.3 unless

Ruby popularizó **unless** como ciudadano de primera clase, y su adopción amplia entre programadores Ruby evidencia que mejora genuinamente la legibilidad en los casos para los que fue

diseñada. Perl admite además una forma postfija (p. ej. `print(x) unless (x == 0)`), descarta porque el cuerpo aparecería antes que la palabra clave y el límite del condicional respecto a lo que lo rodea quedaría sin marcar. Swift (con *guard statements*) y Kotlin ofrecen guardas más potentes que permiten salir anticipadamente de una función; son más expresivas que `unless` pero también más complejas. HULK mantiene una construcción deliberadamente más modesta —solo invierte la condición, sin semántica de salida anticipada—, coherente con su naturaleza de lenguaje basado en expresiones con un único valor de retorno por bloque.

13.4 `loop...while`

Equivalente al `do...while` de C/C++/Java en semántica, con orden léxico distinto: `do...while` introduce dos palabras clave y rompe la convención visual de HULK, mientras que `loop...while` sitúa la condición al final con una sola palabra clave adicional. Pascal usa `repeat...until`, donde la condición de terminación es la negación lógica de la condición de continuación —el bucle termina cuando la condición es *verdadera*—, lo cual puede confundir a programadores acostumbrados al paradigma de C; HULK opta por conservar “continuar mientras `cond`” para alinearse con el `while` precondicional ya existente. En lenguajes funcionales con rasgos imperativos como Scala o Groovy, los bucles postcondicionales son menos frecuentes porque el paradigma favorece la recursión y las comprensiones de listas sobre la iteración explícita; HULK, al ser orientado a objetos con raíces imperativas claras, encaja naturalmente con la tradición de C.

14. Crítica de HULK como diseño de lenguaje

HULK se vende como lenguaje de expresiones con objetos y tipos opcionales. Esa mezcla deja huecos: el `if` no dice cómo se junta con `+`, las funciones no ven el `let` que las rodea, y un `if` con ramas distintas acaba en `Object`. Hay un solo tipo numérico, y dos signos de “igual” que no hacen lo mismo.

14.1 Una orientación a expresiones que no está cerrada

El `if`, el `while`, el `for` y el `let` producen un valor: el `if` entero *vale* algo, no solo dirige el flujo. La especificación muestra `print(if (a% 2 == 0) .even, else .odd)` y lo compara con el ternario de C#, `a% 2 == 0 ? .even : .odd`. En ambos casos el condicional cabe dentro de `print` porque el resultado es una cadena.

Eso ya lo distingue de una *sentencia* como el `if` de Java o de C. Allí el `if` hace algo y no se puede usar como número:

```
int y = 1 + if (c) 2 else 3;    // ilegal en Java
```

En HULK sí hay valor: `let y = if (c) 2 else 3 in print(y)` imprime 2 o 3. El `if` tampoco es un *primario* —un átomo de expresión, como `3` o `f()`— con precedencia escrita frente a `+` y `*`. `2 + 3 * 4` es `2 + (3 * 4)` porque `*` está más apretado que `+`; si el `if` fuera primario, `1 + if (c) 2 else 3` se agruparía igual que `1 + f()`. No es el caso: hay que escribir `1 + (if (c) 2 else 3)`.

Tampoco es un condicional *cerrado*. Ruby marca el borde derecho con `end`:

```
1 + if c then 2 else 3 end
```

El **end** tapa el grupo; no hay duda de dónde acaba. En HULK no hay **end**: el **if** queda abierto a la derecha. Las dos lecturas “naturales” piden reglas opuestas. En una, el **+** vive *dentro* del **else**:

```
if (c) 1 else 2 + 3 // if (c) 1 else (2 + 3) vale 1 o 5
```

En la otra, el **+** vive *fuera* y el **if** es el operando:

```
1 + if (c) 2 else 3 // 1 + (if (c) 2 else 3) vale 3 o 4
```

Un solo **if** abierto no puede ser a la vez la caja que contiene al **+** y la caja que el **+** contiene. El lenguaje tendría que elegir una regla para todos los casos; la otra exigiría paréntesis. HULK no elige.

Python evita el choque con otra forma, **x if c else y**, y un lugar fijo en la tabla de operadores (precedencia propia: el **if** expresión es más flojo que **+**). Así **1 + 2 if c else 3** es **(1 + 2) if c else 3**; si se quiere el condicional como operando hay que escribir **1 + (2 if c else 3)**. No hay dos lecturas en disputa: una gana siempre.

Una jerarquía de infijos es esa escalera de operadores que van en medio de dos operandos (**+**, *****, **@**): **2 + 3 * 4** se arma bajándola. Haskell y ML no meten su **if** en esa escalera. El condicional es sintaxis propia (**if c then e1 else e2**), no un peldaño entre **+** y *****; **if c then 1 else 2 + 3** deja la suma en el **else**, y el **if** como operando de un **+** se agrupa con paréntesis. No mezclan un **if** de C abierto a la derecha con la tabla de infijos.

HULK sí mezcla esas dos ideas: forma del **if** de C —sin **end**— y valor como el ternario, sin decir cómo se junta con **+**, ***** o **@**. Esa omisión es el defecto: cada implementación inventa la asociación. La nuestra —el control encabeza; anidarlo en un operador exige paréntesis— se justifica en la sección siguiente.

14.2 Funciones globales, sin anidamiento ni cierre

Todas las funciones viven en un único espacio de nombres global. No hay funciones anidadas. Una función no captura variables de un **let** exterior: el cuerpo solo ve parámetros, otras funciones globales y constantes predefinidas. Eso obliga a pasar como argumento cualquier dato que, en un lenguaje con cierres, la función vería sola por estar escrita dentro de ese **let**. El patrón **let x = ... in function f() =>x** no existe; hay que escribir **function f(x) =>...** y pasar **x** como argumento en cada llamada.

Python, JavaScript y Scheme definen una función dentro de otra y ven las variables del **let** (o de la función) que las envuelve. En Python, **n** no es parámetro de **sumar**; la función la ve porque está escrita dentro de **hacer**:

```
def hacer(n):
    def sumar(x):
        return x + n
    return sumar
f = hacer(10)
f(3)           # 13: n viaja con sumar
```

C también prohíbe el anidamiento, pero no pretende ser un lenguaje de expresiones con **let**: las variables viven en bloques {}, no en un **let** que produce un valor. HULK toma de C el modelo de funciones (todas globales) y de ML el **let** como expresión, y no define cómo conviven. Esto no existe:

```
let n = 10 in
  function f() => n;    // f no ve n
```

Hay que convertir **n** en parámetro y pasarlo en cada llamada:

```
function f(n) => n;
let n = 10 in print(f(n));
```

El resultado es un **let** que no puede abstraerse: si una expresión usa una variable local, o se copia el cuerpo donde haga falta, o esa variable se vuelve parámetro.

Haríamos distinto permitir *funciones locales* —una **function** escrita dentro de un **let**, no solo al tope del archivo— con *captura léxica*: el cuerpo usa las variables que lo rodean, no solo sus parámetros. Aunque fuera solo de variables que no se reasignan con **:=**, para no pelear con valores que cambian. No hace falta un sistema de lambdas completo (funciones anónimas que son valores, como **(x) => x + n** en JavaScript). Basta con que **function** pueda aparecer donde hoy solo cabe una expresión —el cuerpo tras **in**, una rama de un **if**— y que vea el **let** que la envuelve:

```
let n = 10 in
  function f(x) => x + n in
    print(f(3));    // 13: f ve n
```

Hoy ese hueco solo acepta cosas como **n + 1** o **print(...)**, no una **function**. El costo es guardar, junto a **f**, la tablita de nombres del **let** (un *entorno*). El evaluador ya encadena marcos —cajones **x = 3** → **n = 10** → globales—; lo que falta es no tirar el marco si **f** se usa después de que el **let** “terminó”, y una regla para ese caso.

14.3 Control de flujo: bucles sin salida y huecos de intención

HULK no define **break** ni **continue**. Un **while** o un **for** corre hasta que la condición falla o el iterable se agota. Como ambas construcciones son expresiones, el valor es el del cuerpo (o el de la última iteración), no hay un tipo unidad con el que declarar “este bucle existe por su efecto”, y no hay forma de salir desde un bloque interior antes de que falle la condición de la cabecera, salvo reescribir esa condición.

En C, Java o Python esa salida anticipada es trivial: **break** sale del bucle aunque esté dentro de un **if**.

```
while (true) {
  if (x == 0) break;    // Python/Java/C: sale del while
  x := x - 1;
}
```

En HULK hay que meter esa salida en la condición, porque no existe **break**:

```
while (x != 0) {
  x := x - 1;
}
```

En Rust el bucle también es expresión, y `break` puede *llevar un valor* —el resultado de todo el loop:

```
let n = loop {
    if listo { break 42; }    // el loop vale 42
};
```

HULK toma la tesis “el bucle es expresión” y descarta la operación que, en lenguajes que también la sostienen, permite que esa expresión termine: no hay `break`, con valor ni sin él.

El lenguaje base tampoco nombra la repetición acotada ni la condicional invertida. “Ejecuta esto n veces” se escribe como `while` con contador o como `for` sobre `range`; “hazlo si no se cumple p ” se escribe `if (!p)`. Ninguna de las dos formas dice lo que el programador quiere decir. Ruby, Pascal y Logo tienen esas formas como parte del idioma; HULK no: hay que armarlas con `if` y `while`. Las extensiones `repeat` y `unless` de la sección 11 existen porque el lenguaje base no las nombra, no porque el compilador no supiera implementar un `while`.

El `for` del lenguaje base itera sobre un protocolo de `next/current` (o sobre `range`) sin permitir anotar el tipo del iterador en la cabecera. El programador depende de la inferencia o de un `as` posterior. En un lenguaje de tipado gradual, la cabecera de un bucle es exactamente el sitio donde una anotación es útil: nombra el elemento y permite comprobar estáticamente que el iterable lo produce. Kotlin y C# lo hacen; HULK, en su forma base, no. La variante `for (x : T in expr)` que añadimos no inventa un problema: lo nombra.

14.4 El tipo de un `if` y el polimorfismo por herencia

Las ramas de un `if` se unifican por el ancestro común más bajo. Si ambas producen `Number`, el `if` es `Number`. Si una produce `Number` y la otra `String`, el resultado degenera a `Object`:

```
if (c) 1 else "hola"    // en HULK: Object, no Number ni String
```

A partir de ahí, cualquier uso que no sea `print` o un `as` exige una prueba de tipo. El lenguaje no tiene uniones (`Number | String`) ni un tipo unidad. Una unión nombra las alternativas sin subir a `Object`; TypeScript:

```
let x: number | string = c ? 1 : "hola";
if (typeof x === "number") x + 1; else x.length;
```

Un tipo unidad (`()` en ML/Rust) es un tipo de un solo valor, “no hay resultado”: las dos ramas de un `if` que solo imprimen pueden valer `()` y no tienen que compartir un `Number` o un `String`. Haskell usa un tipo suma con etiquetas:

```
data NumOTexto = UnNum Double | UnTexto String
valor = if c then UnNum 1 else UnTexto "hola"
```

HULK tiene herencia *nominal* —un tipo es subtipo de otro solo si se escribió `inherits`— y un `Object` en la cima. Lo que no comparte un padre más específico se empuja a ese tope: `Number` y `String` solo coinciden en `Object`.

Eso encaja con el otro rasgo del lenguaje base orientado a objetos: el único polimorfismo es por

herencia. Los métodos son virtuales siempre: se ejecuta el de la clase *real* del objeto, no el del tipo con el que se declaró. No hay el `foo` no virtual de C++, pegado al tipo estático.

```
type Animal { foo() => print("animal"); }
type Perro inherits Animal { foo() => print("perro"); }
let a: Animal = new Perro() in a.foo();    // "perro"
```

No hay un protocolo estructural en el subconjunto hasta verificación de tipos: no basta con *tener* `next` y `current` para ganar un nombre `Iterable`. Dos tipos que los implementan son iterables de hecho, pero el lenguaje no les da un nombre común a menos que el programador fabrique un padre (`inherits Iterable`). Java exige declararlo (`class Cola implements Iterable {...}`); si no escribes `implements`, no es `Iterable` aunque tenga los métodos. Go no lo exige: si `Cola` tiene `Next` y `Current`, ya es `Iter`.

```
type Iter interface { Next() bool; Current() any } // Go: implícito
```

HULK espera a una capa posterior (protocolos) para decirlo, y hasta entonces obliga a la jerarquía.

Haríamos distinto unificar las ramas del `if` con un tipo unión cuando no hay ancestro útil, y tratar el protocolo de iteración como parte del lenguaje base —no como documentación informal de dos métodos— para que `for` no dependa de un padre inventado. El `Object` como cajón de un `if` heterogéneo es la decisión que más ruido introduce en programas que, por lo demás, son de expresiones pequeñas.

14.5 Un solo Number

HULK no distingue entero de flotante. El literal `1`, el resultado de `1 / 2` y el contador de un `for` sobre `range` son el mismo tipo. La división no es la de C ($1/2 \rightarrow 0$): aquí `1 / 2` es `0.5`. Un índice de bucle y una medida continua no se pueden separar en el sistema de tipos.

```
let i = 0 in           // índice? o medida?
  1 / 2                // 0.5, no 0
for (k in range(0, 3))
  print(k);           // k es Number, como 0.5
```

Python 3 y JavaScript también unifican. C y Rust no: `int` frente a `double`, `i32` frente a `f64`. Lo que se gana es un lenguaje chico: una aritmética, sin conversiones de índice a flotante. Lo que se pierde es poder decir “esta variable es un índice”: no hay desbordamiento de entero aparte del error de flotante, `0.1 + 0.2` no es `0.3`, y `range` cuenta con el mismo tipo con el que se divide. Un `Integer` aparte —aunque solo para cabeceras de `for` y tamaños— cerraría el hueco sin complicar el resto. Para un lenguaje didáctico la simplicidad se defiende; como diseño de tipos, queda abierto.

14.6 `= y :=`

El `let` introduce un nombre con `=`. Reasignar usa `:=`: el nombre ya existía. Fuera de un `let`, `x = 1` es un error. El tropiezo habitual es escribir la reasignación con el signo del `let`:

```
let x = 1 in {
  x := x + 1;    // bien: cambia el valor
```

```
x = x + 1;          // ilegal: parece let, no lo es
}
```

Python usa `=` para las dos cosas (`x = 1` declara o cambia el valor). C también, y no tiene `let`. ML parece al `let` de HULK y reserva otra forma para mutar. HULK toma las dos ideas y las deja en el mismo programa, con dos signos fáciles de mezclar. El compilador puede explicar el error; el diseño obliga a recordarlo en cada línea. Un solo `=` según el contexto —el nombre es nuevo o ya existe—, o prohibir la reasignación, cerraría el hueco. No es una extensión: es el idioma base.

15. El `if` y la cadena de operadores

La omisión anterior hay que cerrarla en una gramática concreta. Sin un `end`, las dos lecturas naturales del `if` frente a `+` no caben a la vez. Elegimos una: el control encabeza. `if (c) 1 else 2 + 3` significa `if (c) 1 else (2 + 3)`. En cambio `1 + if (c) 2 else 3` no se acepta; hay que escribir `1 + (if (c) 2 else 3)`. Un `if` sí puede ser argumento de una llamada —`print(if (c) 2 else 3)`— porque los argumentos son expresiones plenas, no operandos de un infijo. La especificación no impone esa frontera: muestra el `if` dentro de `print` y nunca como operando de una suma, y no obliga a aceptar la segunda forma. La frontera es nuestra.

15.1 Qué se descartó

La alternativa inmediata es colocar `IfExpr` en `Primary`. Entonces `1 + if (c) 2 else 3` parsearía, y `if (c) 1 else 2 + 3` se agruparía como `(if (c) 1 else 2) + 3`. Eso invierte la asociación que privilegiamos. Si `IfExpr` siguiera siendo además alternativa de `Expr`, el token `IF` estaría en `FIRST(IfExpr)` y en `FIRST(AssignExpr)`, y habría un conflicto `FIRST/FIRST` en `Expr`, no en la producción de la suma. Dejar el `if` únicamente como primario elimina el conflicto y cambia el `else`. Mantenerlo en `Expr` y exigir paréntesis conserva la asociación. `LL(1)` admite las dos gramáticas; no es que la clase del parser “no pueda” aceptar el operando.

Tampoco tomamos el camino de Ruby (añadir un cerrador) ni el de C (otra construcción, `?:`, cuya precedencia es más baja que la de `+`: `1 + c ? 2 : 3` no agrupa como `1 + (c ? 2 : 3)`). El `if` de HULK sigue siendo `if`; los paréntesis no cambian su semántica, cambian el agrupamiento visible. `x + if (c) a else b` obliga a decidir, hacia atrás desde el `+`, dónde termina la suma. `x + (if (c) a else b)` no.

El mismo criterio ya había descartado la forma postfija de Perl al diseñar `unless`. Perl admite `print(x) unless (x == 0)`: el cuerpo aparece antes que la palabra clave y el límite con lo que rodea al condicional queda sin marcar. No la adoptamos. Aquí ocurre lo mismo: no se admite una construcción de control cuyo borde respecto a los operadores vecinos no esté delimitado. En un caso el marcador que faltaría es un `end` o un paréntesis; en el otro, el orden palabra clave–cuerpo. No se compra una comodidad sintáctica que mezcle el control con los infijos sin delimitador.

En la gramática esa decisión se lee así. `Expr` admite el `if`, el `let`, el `while`, el `with`, el `case` y las extensiones `unless`, `repeat` y `loop...while` como alternativas, junto a la cadena que baja desde la asignación hasta los literales. Los operandos de `+` no son `Expr`: `AddExprTail` exige `MulExpr`. Por eso el operando derecho de una suma no puede empezar por `if`. El anidamiento explícito es `Primary -> LPAREN Expr RPAREN`.

15.2 Cómo se hace visible el rechazo

El parser no emite un *syntax error* genérico. Si tras + llega **if** (o **let**, **while**, **with**, ...) el mensaje, escrito en `Parser/syntax/ll1_parser.cpp`, es:

El operador '+' solo combina expresiones multiplicativas (literales, identificadores, llamadas, agrupación con parentesis, etc.); 'if' inicia una expresión de otro tipo y no puede usarse como operando aquí

Nombra el operador, la categoría que sí combina y la palabra clave que no es operando. Los paréntesis aparecen como forma válida de operando, que es la corrección. Las pruebas en `Parser/tests/expr_precede` rechazan `x + if (a) b else c` y los análogos con **let**, **while** y **with**. El caso **let** exige además el fragmento “solo combina expresiones multiplicativas”; los de **if**, **while** y **with** exigen que el análisis falle. No pertenecen al esqueleto del curso: si se relajara la frontera, dejarían de pasar.

El costo es un par de paréntesis en `print(1 + (if (x > 0) x else 0))`. El beneficio es una sola convención, un **else** con asociación estable, y un diagnóstico que dice qué está mal en lugar de un token inesperado.

16. Decisiones de diseño y compromisos

16.1 Azúcar sintáctica con implementación nativa

Las tres extensiones secundarias no añaden poder computacional: expresan patrones ya posibles con **if** y **while**. El **for** tipado añade verificación estática explícita. El valor académico está en la expresividad y en la integración completa en el compilador, no en nuevas primitivas.

Implementar nodos nativos en lugar de desazucarar el árbol implica más código en tres visitantes, pero evita una fase extra, permite optimizar **range** y facilita pruebas de extremo a extremo comparando intérprete y ejecutable nativo.

16.2 Compatibilidad y uniformidad

Todas las extensiones son compatibles con programas HULK previos: las palabras clave nuevas no colisionan con identificadores existentes y la forma **for** (**x in expr**) se conserva. Las construcciones siguen el patrón visual del lenguaje: condiciones entre paréntesis, cuerpos como expresiones o bloques.

16.3 Principio de menor sorpresa

unless invierte la condición; **repeat** repite; **loop...while** ejecuta el cuerpo antes de comprobar; **for** : T solo restringe tipos cuando el programador lo indica.

17. Validación y pruebas

17.1 Estrategia general

El criterio principal de corrección para la generación de código es:

```
./hulk programa.hulk && ./output ≡ ./hulk programa.hulk -interpret
```

17.2 Conjuntos de prueba de las extensiones

- Pruebas de extremo a extremo: `tests/extensions/manifest.tsv` (por ejemplo `for_typed.hulk`, `repeat_zero.hulk`, `unless_else.hulk`, `loop_while_once.hulk`).
- Evaluación: `tests/eval/valid/43_v9_unless.hulk`, `44_v9_repeat.hulk`, `45_v9_loop_while.hulk`.
- Generación LLVM: `tests/llvm/valid/83_l8_unless.hulk-92_l9_loop_while_multi.hulk`.
- Verificación semántica: `72_i7_for_typed.hulk` (válido), `64_i7_for_type_mismatch.hulk` (inválido), `54_t_unless_non_boolean.hulk`, `61_g_repeat_non_string.hulk`.

Los casos anteriores cubren tanto usos esperados como casos límite. Para el `for` tipado se valida la iteración sobre rangos numéricos con y sin anotación, la iteración sobre colecciones personalizadas que implementan `Iterable`, y el caso de error en que el tipo anotado no conforma al devuelto por `current()` (`64_i7_for_type_mismatch.hulk`). Para `repeat` se prueba la repetición con argumento positivo, cero y negativo, y la anidación dentro de otras construcciones de control de flujo. Para `unless` se cubren las cuatro combinaciones de forma sin/con `else` y condición verdadera/falsa, verificando en cada caso que el tipo de retorno se infiere como el ancestro común más bajo de ambas ramas. Para `loop...while` se prueba el caso en que la condición es falsa desde el inicio (el cuerpo igualmente se ejecuta una vez), el caso de múltiples iteraciones, y la anidación con otras construcciones.

17.3 Objetivos del Makefile

`make test_semantic`, `make test_eval`, `make test_llvm` y `make test_llvm_fixtures` automatizan la regresión. La integración continua del repositorio en MatCom ejecutará además las pruebas estándar del curso cuando se publiquen.

18. Limitaciones y trabajo futuro

- `repeat` no expone índice de iteración al programador.
- `unless` no admite `elif`.
- No hay `break/continue` en HULK (limitación del lenguaje base).
- El `for` tipado verifica conformidad de tipos, no igualdad exacta con `current()`.
- Características posteriores a la verificación de tipos en la documentación oficial (protocolos formales, vectores, funciones anónimas, macros) quedan fuera del alcance actual.
- Una fase opcional de desazucarado del árbol sintáctico abstracto podría unificar la implementación en el futuro, pero no es necesaria para la semántica actual.

19. Conclusiones

Hemos construido un compilador HULK completo que recorre análisis léxico (Flex), análisis sintáctico LL(1), verificación semántica con inferencia multipasada, generación LLVM y un intérprete de validación. El compilador implementa el núcleo del lenguaje hasta verificación de tipos y añade un conjunto coherente de cuatro extensiones de control de flujo e iteración.

Técnicamente, las extensiones muestran cómo enriquecer la expresividad de un lenguaje didáctico integrando cambios en léxico, gramática, árbol sintáctico abstracto, semántica y ambos motores de ejecución. La semántica formal se expresa por equivalencia con construcciones base; la implementación mantiene nodos dedicados y visitas equivalentes en el verificador semántico, el evaluador y LLVM.

Desde el diseño de lenguajes, el trabajo ilustra compatibilidad regresiva, tipado gradual opcional en el `for`, y comparación con construcciones de Ruby, Kotlin, C# y C. HULK, mirado como lenguaje y no como compilador, deja sin cerrar cómo se compone el control con los operadores, no da entorno léxico a las funciones y unifica ramas heterogéneas de un `if` empujándolas a `Object`; la asociación que adoptamos para el `if` frente a `+` es una convención nuestra, no un mandato de la especificación. El informe, junto con el repositorio y los conjuntos de prueba, documenta la arquitectura del compilador, las extensiones y esa crítica.